Review package: browser lifetime + stop rule S4

Branch	`fix/browser-lifetime-and-stop-rule` (pushed to origin)
Base	`b676c2d`
Release candidate	**`9a1b931`** (code + tests). It contains the accepted browser-lifetime fix `f2288ce`, the harness commit `caaa191`, and the release-item commit `9a1b931` (durable stop-rule cancel, identified/allowlisted integration helper). The commits that add these docs sit on top of it and change no code.
Patch	`fix-browser-lifetime-and-stop-rule.patch` = `git format-patch b676c2d..9a1b931 -- . ':(exclude)deliverables'` (4 code commits; verified to reproduce the `9a1b931` tree when applied to `b676c2d`)
Gates	**The browser fix is active as soon as this is deployed.** Production already runs with `WORKER_ISOLATION_ENABLED=true`, and the fix applies in isolated job children. There is no separate switch for it. **Only the stop rule is inert by default**: it does nothing unless `BATCH_STOP_RULE_FAILED_JOBS` is set to an integer > 0.

Contents

Document	What it contains
BRANCH_DIFF.md	Commits, files, what changed in `processor.ts` / `worker.ts` / `storage.ts` / `batch-stop-rule.ts`, gates, review focus
TEST_RESULTS.md	Before/after repro (unlimited and Tasks-Max=400), new tests, regression suites, integration suite with process identities, real-PG stop-rule test, tsc, what is not verified
evidence/batch1276-url-reconciliation.md	Batch 1276: failed, then recovered, then residual gap per company. **Batch 1276 is a failed/degraded run** (see below).
`evidence/browser-lifetime-candidate/`	Raw repro JSON and logs for the four worktree runs, plus `wt-run.sh`
`evidence/regression-caaa191/`	Suite logs, tsc outputs (base and `caaa191`), integration logs for the previous candidate
`evidence/release-items/`	Release-item evidence for `9a1b931`: stop-rule unit and real-PG logs, three integration runs with process identities, `tsc.txt`, and `regression/` (all suite logs re-run on `9a1b931`)

Release items (reviewer's list) — outcomes on `9a1b931`

1. **Durable stop-rule cancel: done, tested.**
   - The cancel sequence now runs the flag first (local + Redis notification, synchronous). It then starts the PG cancel and the queue purge together, so the PG cancel is never gated behind the purge.
   - The purge is bounded (10 s). The snapshot runs only after the PG cancel has committed, and only on the path that actually transitioned the batch. It is bounded (30 s), and its rows are marked `rejected` with `cancelled/partial snapshot … not comprehensive`.
   - Unit tests with never-resolving purge/snapshot fakes show the PG cancel completing in < 100 ms and the sequence settling within its bounds: 20/20 pass with local PG, 19 + 1 skip without it. The rule is still inert at threshold 0/unset (test 9).
2. **Extra integration process: identified, allowlisted by exe + cmdline. The zero-descendant check was not relaxed.**
   - It is tsx's esbuild transform service: exe `node_modules/.pnpm/@esbuild+linux-x64@0.28.0/.../bin/esbuild`, cmdline `<exe> --service=0.28.0 --ping`, parent = the test harness process.
   - It is spawned lazily when tsx compiles uncached TypeScript. In the cold-cache run it already existed before any job ran; with a warm cache it does not appear at all.
   - It is test toolchain, not app work (production runs compiled JS without tsx). It exits with the har-

ness.
- Final run: 12/12 PASS, with 0 counted descendants after completion and after `stopWorker`.

3. **Real-PG stop-rule test: PASS 17/17**, run on a throwaway database on a disposable local PG container, which was dropped afterwards.
- Exercises `COUNT_BATCH_FAILURES_SQL`, the real `failJob` / `cancelBatchRun` / `claimJob` /BullMQ purge/Redis flag, and two concurrent failure paths racing to trip the rule.
- Outcome: durably cancelled, exactly one PG transition, exactly one snapshot (rejected, partial), zero retries, and no re-claim.
- Production `companyiq_v3` and the hosted `default` DB were not touched.

Headline results (browser fix, unchanged from the accepted review)

Live browsers.

- With 1 browser permit, the base keeps up to **4** Chromium instances alive (one per child), and the candidate keeps **1**.
- Threads fell from 550 to 205 and summed RSS from 4.9 GB to 2.0 GB. Wall time, admission wait and permit hold did not change in this workload.

Under a 400-task cgroup cap (TasksMax=400).

- The base failed **4/12** jobs. Those documents were never fetched, after Chromium launch failures and protocol timeouts, and the run took 3.9× as long.
- The candidate ran **12/12** with unchanged timing.

Ordering. `browser-close-order.test.ts` (4/4) shows that Chromium is gone before the permit returns on the success, fetch-failure, cancellation and child-death paths. A mutation check showed that releasing the permit first makes these tests fail.

Regressions on `9a1b931`.

- Every suite was re-run and exits 0, including `lifecycle-fence.db` (165/165 on real local PG/Redis, which covers concurrent `cancelBatchRun`).
- tsc: 62 errors, an error set identical to the base. One new error (a test variable typed `Promise<void>` for `cancelBatchRun`'s new `Promise<boolean>`) was found and fixed before commit.
- Integration (real PG/Redis/BullMQ/Chromium): **12/12 PASS**. One earlier attempt failed 6 checks, but that was caused by my own first real-PG stop-rule test, not by the product. It left its Redis cancel flag `cancelled:batch:9` behind; the next integration run reused batch id 9, so its batch was pre-cancelled and never claimed. The flag was deleted and the test's teardown now removes it. The log is kept as `integration-identity-attempt1-redis-polluted.log`.

Stop rule. In the batch-1276 shape, the rule off means 13 jobs exhaust retries (39 failed attempts). With `BATCH_STOP_RULE_FAILED_JOBS=2`, the batch is cancelled at the 2nd failed job, before any retry is exhausted.

Batch 1276: a failed/degraded run

Batch 1276 is a failed/degraded run. Its completed companies must **not** be treated as confirmed-comprehensive results. They were scored on a corpus with a known residual gap.

- 95 failed URLs: 26 recovered by fallback, 20 ok in DB without a logged recovery, and **49 URLs never recovered** (12.5% of the attempted corpus).

- That includes **10 of Southern's 27** URLs (37%), all after browser-launch failures. That is the clearest coverage loss consistent with the browser-lifetime defect. It is an inference from log tags, not a measured causal link.

Claims kept narrow

- **SIGABRT not reproduced.** The production SIGABRT signature was **not** reproduced locally. Locally we reproduced a launch-failure / starvation mode under a task cap.
- **What the fix is shown to do.** It removes the browser accumulation demonstrated locally (live Chromium bounded by permits, not by children), and the extraction failures that came with it in that constrained repro. **Production must still show whether it eliminates the child-start failures** seen there.
- **Overhead.** Overhead was negligible **in this local workload only** (wall time, admission wait and permit hold unchanged). This is not a general zero-overhead claim. Closing and relaunching Chromium per permit cycle has a cost that depends on fetch mix and concurrency.
- **Environment.** Not run on Railway or at production settings. No real web fetches or LLM providers were used.

Rollback procedure

1. **Rollback = revert the deploy.** Redeploy the previous production image/commit: `b676c2d`, or whatever the production deploy history shows as the deploy immediately before this one. Confirm the commit there before redeploying. This removes both the browser fix and the stop rule.
2. **To turn off only the stop rule** (keeping the browser fix): unset `BATCH_STOP_RULE_FAILED_JOBS` or set it to `0`. The value is read from the process environment, so it takes effect once the worker process restarts with the new value.
3. **Do NOT disable `WORKER_ISOLATION_ENABLED` as a rollback.** That would switch production onto the in-process path, which is a different, larger change than this release, and it is not a rollback of this release. Reverting the deploy is the rollback.
4. No data migration to undo. The release adds no schema change. Stop-rule snapshots, if any were taken, are already marked `rejected` / cancelled/partial and can stay.